Universidad Sergio Arboleda

Procesamiento de Lenguaje Natural

Parcial 1 NLP

Estudiantes:

Santiago Diaz
Juan Camilo Gallardo
Alan Osorio

Profesor:

Juan Pablo Ospina

9 de marzo de 2026

1 Análisis de Ablación

El análisis de ablación cuantifica la contribución de cada componente del pipeline al desempeño final del sistema. Se toma como referencia el baseline definido en la sección anterior (Multinomial Naive Bayes con codificación Bag-of-Words) y se varía un único componente a la vez, manteniendo fijos los demás.

Justificación de Métricas: Para la evaluación de los modelos se han seleccionado el **F1 macro** y el **ROC-AUC** como métricas principales. El F1 macro se elige porque calcula la media armónica entre precisión y exhaustividad de forma independiente para cada clase y luego promedia los resultados, lo que penaliza al modelo si tiene un mal desempeño en una clase específica, asegurando que ambas clases (positiva y negativa) se clasifiquen correctamente sin importar el desbalance de los datos. Por su parte, el ROC-AUC se incluye para medir la capacidad general del modelo para distinguir entre las diferentes clases frente a distintos umbrales de decisión, ofreciendo una visión integral del rendimiento predictivo más allá del accuracy tradicional.

1.1 Baseline de Referencia

Table 1: Métricas del modelo baseline (MultinomialNB, $\alpha = 5.0$, BoW unigrama).

Conjunto	**Accuracy**	**F1 macro**	**ROC-AUC**
Train	0.7890	0.7890	0.8620
Validación	0.7810	0.7809	0.8534
Test	0.7827	0.7826	0.8552

1.2 A. Preprocesamiento con spaCy

Se utilizó el modelo `en_core_web_lg` de spaCy manteniendo fijo el orden del pipeline: tokenizar → normalizar → lematizar → filtrar. Se ejecutaron cinco experimentos variando un único componente respecto al baseline en cada caso.

Table 2: Configuración de cada experimento de preprocesamiento.

Exp.	**Descripción**	**Lemma**	**Stopwords**	**Emojis**	**Punt.**	**Alarg.**
Baseline	Referencia (sin spaCy)	off	drop	drop	drop	off
Exp 1	Con lematización	on	drop	drop	drop	on
Exp 2	Con puntuación	on	drop	drop	keep	on
Exp 3	Con stopwords	on	keep	drop	drop	on
Exp 4	Emojis traducidos	on	drop	transl.	drop	on
Exp 5	Sin norm. alarg.	on	drop	drop	drop	off

Table 3: Resultados de los experimentos de preprocesamiento sobre el conjunto de prueba.

Exp.	**Descripción**	**Accuracy**	**F1 macro**	**ROC-AUC**
Baseline	Referencia	0.7827	0.7826	0.8552
Exp 1	Con lematización	0.7600	0.7600	0.8353
Exp 2	Con puntuación	0.7582	0.7582	0.8313
Exp 3	Con stopwords	0.7651	0.7651	0.8410
Exp 4	Emojis traducidos	0.7826	0.7826	0.8552
Exp 5	Sin norm. alarg.	0.7835	0.7835	0.8559
Mejor	**Exp 5**	**0.7835**	**0.7835**	**0.8559**

A partir de los resultados observados, se pueden extraer las siguientes conclusiones para cada variación del preprocesamiento:

- **Exp 1 (Con lematización):** El rendimiento disminuyó frente al baseline, lo que sugiere que en el contexto de Twitter (lenguaje informal), la forma original conjugada de las palabras aporta información valiosa para detectar el sentimiento que se pierde al llevarlas a su lema.
- **Exp 2 (Con puntuación):** Mantener los signos de puntuación introdujo ruido, empeorando todas las métricas. Esto indica que no representan una señal determinante o están demasiado desestructurados en los datos.
- **Exp 3 (Con stopwords):** Conservar palabras vacías diluyó la relevancia de los términos con verdadera carga emocional, provocando una caída en el desempeño general.
- **Exp 4 (Emojis traducidos):** El resultado fue prácticamente idéntico al del baseline (donde se descartaron), indicando que la traducción de emojis a texto no aportó un valor discriminativo significativo para el modelo BoW unigrama ya que al momento de hacer la transformacion a parquet los emojis quedan en formato de simbolos y luego al hacer la limpieza simplemente se eliminan.
- **Exp 5 (Sin norm. alarg.):** Fue la única configuración que mejoró el baseline. Esto demuestra que los alargamientos expresivos en redes sociales (por ejemplo, "hellooooo" o "yeees") funcionan como un indicador útil de la intensidad del sentimiento, y normalizarlos resta precisión.

Con base en este análisis, se selecciona la configuración **Exp 5** como la más favorable, con una mejora de **+0.09 pp** en F1 macro respecto al baseline. Esta configuración se fija para los experimentos de codificación.

1.3 B. Codificación

Utilizando el preprocesamiento óptimo y el mismo modelo baseline (MultinomialNB), se comparan tres esquemas de codificación.

Table 4: Resultados según esquema de codificación sobre el conjunto de prueba.

Codificación	**n-gram**	**Accuracy**	**F1 macro**	**ROC-AUC**
BoW (baseline)	Unigrama	0.7827	0.7826	0.8552
TF-IDF	Unigrama	0.7715	0.7715	0.8406
TF-IDF	Bigrama	0.8021	0.8020	0.8833
Mejor	**TF-IDF**	**0.8021**	**0.8020**	**0.8833**

La codificación **TF-IDF** con n-gramas **Bigrama** ofrece el mejor desempeño y se fija para los experimentos de modelado. Esto se debe a que, por un lado, la transformación TF-IDF pondera la importancia de los términos, penalizando palabras muy frecuentes en todo el corpus que no aportan significado (ruido) y destacando aquellas que son verdaderamente discriminativas para cada clase. Por otro lado, la inclusión de **bigramas** permite al modelo capturar el contexto local y relaciones secuenciales entre palabras (como negaciones del tipo "no bueno" o frases compuestas), algo que la evaluación aislada de unigramas pierde por completo. Esta combinación de ponderación y contexto enriquece la representación del texto, logrando el aumento significativo en las métricas de evaluación.

1.4 C. Modelos

Con el preprocesamiento y codificación óptimos se evalúan los modelos de distintas familias utilizando las métricas obtenidas.

Table 5: Comparación de modelos sobre el conjunto de prueba.

Modelo	**Familia**	**Accuracy**	**F1 macro**	**ROC-AUC**
MultinomialNB (baseline)	Probabilístico	0.7827	0.7826	0.8552
LogisticRegression	Paramétrico	0.8232	0.8232	0.9025
LinearRegression	Paramétrico	0.8125	0.8124	0.8918
LinearDiscriminantAnalysis	Paramétrico	0.7313	0.7312	0.8092
DecisionTreeClassifier	Árboles	0.6474	0.6471	0.7097
RandomForestClassifier	Ensambles	0.7324	0.7322	0.8073
XGBoost	Ensambles	0.7393	0.7385	0.8188
FFNN	Deep Learning	0.8198	0.8198	0.9024
DeepNeuralNetwork	Deep Learning	0.7773	0.7773	0.8608
MLPClassifier	Deep Learning	0.7709	0.7709	0.8549
Mejor	**Regresión Log.**	**0.8232**	**0.8232**	**0.9025**

Los resultados demuestran que la **Regresión Logística** obtiene el mejor desempeño general, estrechamente seguida por la red neuronal **FFNN** y la **LinearRegression**. El éxito de los modelos paramétricos (especialmente la Regresión Logística) sobre representaciones TF-IDF se explica porque este espacio vectorial genera matrices altamente dispersas (sparse matrices) y de altísima dimensionalidad, en las cuales las relaciones entre las características y las clases suelen ser linealmente separables. Por el contrario, los modelos basados en partición de espacios (DecisionTree, RandomForest y XGBoost) mostraron un rendimiento considerablemente menor; estos algoritmos sufren ante datos extremadamente dispersos y multidimensionales, ya que no pueden encontrar cortes eficientes en las variables sin sobreajustarse o volverse ineficientes computacionalmente. Aunque los enfoques de Deep Learning (como MLP y DNN) logran capturar relaciones no lineales, en este contexto de texto estructurado por TF-IDF, no superan la simplicidad, rapidez y robustez de una regresión logística bien regularizada.

1.5 D. Comparación con DistilBERT (Hugging Face)

Como referencia de desempeño esperado se evalúa el pipeline `sentiment-analysis` de Hugging Face, basado en DistilBERT fine-tuned en SST-2, sobre el conjunto de prueba. Este modelo no fue entrenado con datos de Twitter, lo cual debe considerarse al interpretar los resultados.

Table 6: Comparación baseline, mejor modelo propio y DistilBERT (Test).

Modelo	**Accuracy**	**F1 macro**	**ROC-AUC**	**Lat. (s/muestra)**
MultinomialNB baseline	0.7827	0.7826	0.8552	–
Regresión Logística	0.8232	0.8232	0.9025	–
DistilBERT SST-2	0.7109	0.7081	0.7934	–

Nota: *DistilBERT fue entrenado sobre SST-2 (reseñas de cine), cuyo lenguaje es formal y estructurado. El lenguaje de Twitter contiene abreviaciones, emojis y ruido que difieren de ese dominio. Se espera que el modelo propio, entrenado directamente sobre Sentiment140, supere a DistilBERT en este conjunto de prueba a pesar de su menor capacidad paramétrica.*

2 Despliegue del Modelo mediante FastAPI

Para poner en producción el modelo seleccionado (Regresión Logística con TF-IDF Bigrama), se construyó una API RESTful utilizando **FastAPI**. Esta API permite realizar inferencias en tiempo real y consultar de manera interactiva los metadatos y resultados de todos los experimentos realizados en este proyecto.

La documentación interactiva (Swagger UI) se encuentra alojada en la siguiente dirección:

`http://54.84.197.7:8000/docs`

A continuación se describen los *endpoints* principales disponibles en la API:

2.1 Endpoint de Predicción (`/predict`)

Este endpoint recibe peticiones mediante el método `POST` para clasificar el sentimiento de nuevos textos. Está diseñado para soportar tanto inferencia individual (procesando un solo texto) como inferencia por lotes (batch processing). Se envía un objeto JSON con el texto objetivo y el modelo retorna la polaridad predicha y la confianza de la predicción.

A continuación se muestra cómo ejecutarlo de manera sencilla desde la terminal utilizando `curl`:

```
BASE="http://54.84.197.7:8000"

# Individual
curl -X POST $BASE/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "I absolutely love this product!"}'

# Batch (L neas divididas para evitar desbordes visuales)
curl -X POST $BASE/predict \
  -H "Content-Type: application/json" \
  -d '{
        "texts": [
          "I love this!",
          "This is terrible.",
          "Not bad at all.",
          "Worst experience ever."
        ]
      }'
```

2.2 Endpoints de Metadatos y Desempeño

Se desarrollaron *endpoints* bajo el método `GET` que exponen directamente los resultados del análisis de ablación y las características del modelo ganador. Estos recursos son útiles para consultar las capacidades del pipeline desplegado sin necesidad de revisar el código fuente.

- `/model_info`: Devuelve un JSON detallado con la configuración exacta del modelo seleccionado en producción. Incluye el tipo de modelo, parámetros de vectorización (TF-IDF), hiperparámetros de la regresión logística (ej. `C=1`, `solver="saga"`), y el autor de dicho experimento (Juan Camilo Gallardo).

- `/comparison`: Retorna una estructura JSON que documenta la comparación directa de las métricas obtenidas entre el modelo ganador (LR

+ TF-IDF Bigrama) y el modelo preentrenado de HuggingFace (DistilBERT). Contiene el F1-macro y ROC-AUC alcanzados por cada modelo sobre el conjunto de test.

- `/work_distribution`: Este endpoint detalla cómo se dividieron las pruebas entre los miembros del equipo. Por cada integrante (Alan Osorio, Juan Camilo Gallardo, Santiago Diaz), devuelve un listado de los modelos ejecutados, la cantidad de experimentos y la métrica F1 de su mejor modelo respectivo.

- `/ablation_summary`: Expone la información consolidada de los 18 experimentos del proyecto en formato estructurado (JSON). Incluye el nombre de cada prueba, el autor, el esquema de vectorización, el clasificador y sus respectivas métricas (F1 y AUC).

2.3 Endpoints de Descarga de Archivos

Adicionalmente, se configuraron endpoints diseñados específicamente para descargar recursos relacionados con el análisis experimental. Al ejecutar una petición `GET` sobre ellos, el servidor responde con el recurso para ser guardado localmente:

- `/ablation_table`: Genera y descarga un archivo de texto plano (`ablation_study.txt`) que contiene una tabla ASCII consolidada con todos los resultados, lo cual facilita su lectura y exportación.

- `/work_distribution_table`: Retorna un archivo de texto descargable con la distribución de experimentos entre los integrantes del grupo.

- `/ablation_chart`: Al acceder a esta ruta, se renderiza y descarga directamente un archivo de imagen PNG que grafica el F1-macro alcanzado por cada uno de los experimentos listados.

3 Conclusiones Finales

A partir del desarrollo del pipeline de clasificación de sentimientos y el exhaustivo análisis de ablación realizado, se pueden extraer las siguientes conclusiones fundamentales:

- **Consolidación del Modelo Óptimo:** Se determinó que el mejor modelo para esta tarea es la **Regresión Logística combinada con vectorización TF-IDF de Bigramas**. Se llegó a este resultado a través de una metodología incremental estricta: partimos de un *baseline* básico (BoW), optimizamos el preprocesamiento descubriendo que conservar los alargamientos de palabras es clave para este dominio, elevamos la complejidad de codificación a TF-IDF Bigrama para capturar contextos, y finalmente evaluamos este pipeline frente a 9 algoritmos distintos. Es

el mejor modelo porque logra procesar de manera excepcionalmente eficiente la alta dimensionalidad y dispersión (sparsity) de las matrices de texto, encontrando relaciones lineales robustas. Esto le permitió alcanzar la métrica más alta (F1-macro de 0.8232), superando a complejos ensambles basados en árboles y logrando una altísima eficiencia de inferencia, haciéndolo ideal para desplegar en la API.

- **Importancia del contexto de los datos:** Las técnicas tradicionales de normalización en procesamiento de lenguaje natural (como forzar la lematización) no siempre son adecuadas para todos los dominios. En escenarios informales como Twitter, los errores ortográficos deliberados, conjugaciones particulares y alargamientos expresivos (por ejemplo, "siii") contienen una valiosa carga emocional predictiva que se pierde al aplicar una limpieza excesiva.

- **Evolución de la codificación de texto:** La transición de un modelo unigrama simple de Bag-of-Words a una representación TF-IDF enriquecida con bigramas probó ser la mejora individual más potente del proceso. Esto subraya la necesidad técnica de amortiguar el ruido del vocabulario general y de retener la secuencialidad básica del texto (necesaria para capturar contextos y negaciones).

- **El algoritmo adecuado para el espacio adecuado:** Ante un espacio de características denso, de alta dimensionalidad y extremadamente disperso (matriz TF-IDF), los modelos lineales/paramétricos como la Regresión Logística brillan por su alta eficiencia y su excelente capacidad para trazar hiperplanos de separación de clases. Por el contrario, los ensambles basados en árboles resultaron ser subóptimos para lidiar eficientemente con este nivel de *sparsity*.

- **Adaptación de dominio vs. Modelos de estado del arte:** La comparación final demuestra una de las mayores lecciones de NLP aplicado en la actualidad: un modelo "simple" entrenado a medida en el dominio objetivo (nuestro pipeline de LogReg en datos de Twitter) es capaz de superar contundentemente a un modelo Transformer gigante pero descontextualizado (DistilBERT entrenado en SST-2). La adaptación al dominio es más determinante para el rendimiento final que la complejidad paramétrica de la arquitectura.